NG-INDIA 2026

Angular SSR: Rendering, Hydration, and the Hidden Tradeoffs

Behind the scenes of SSR and how to decide when the complexity is worth it.

Aprajita Verma
Frontend Architect, Mycom

TALK OVERVIEW

Agenda

1. **The Claim vs The Reality**
   Four vitals. Four honest answers.
2. **What You're Actually Buying**
   Two runtimes. Per-route decisions.
3. **The Technical Complexity**
   18 problems. Four buckets.
4. **The Full Cost Surface**
   The costs that compound over time.
5. **The ROI Question**
   When the investment pays off.
6. **Three Questions**
   One slide. Walks out with them.

THE PREMISE

"Enable SSR. Improve your Core Web Vitals."

Let's be precise about what that actually means.

SEGMENT 1 — THE CLAIM VS THE REALITY

Four vitals. Four different answers.

LCP — LARGEST CONTENTFUL PAINT

Conditionally improves

Only if TTFB + server render time < JS parse + execution time. Not guaranteed. A slow server with no caching will **hurt** LCP.

CLS — CUMULATIVE LAYOUT SHIFT

May get worse

Hydration mismatches cause layout shifts that wouldn't exist in pure CSR. `NG0500` = Angular discards server DOM = layout shift.

INP — INTERACTION TO NEXT PAINT

SSR doesn't touch it

Interaction is entirely client-side. `@defer` + `withIncrementalHydration()` move INP — not SSR itself.

TTFB — TIME TO FIRST BYTE

Worse without caching

You're adding a server render on top of network time. Every single request. Caching is not optional — it's what makes SSR viable.

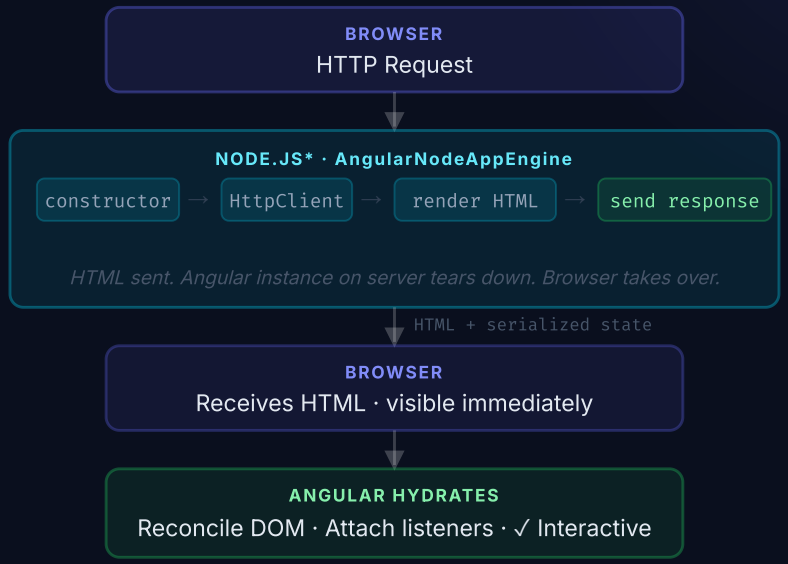
SEGMENT 2 — WHAT YOU'RE ACTUALLY BUYING

You now have two apps.

- **Node.js* (Server)**
  `AngularNodeAppEngine` boots your app, executes components, fetches data, produces HTML — sends the response, then the server instance tears down.

- **Browser (Client)**
  Angular boots again. Hydration reconciles the server DOM against the client render. Two executions. One user.

- Everything you write runs in both worlds unless you explicitly tell it not to.



* also: AWS Lambda · Vercel Functions · Cloudflare Workers (edge) · Netlify Functions · Azure Functions

SEGMENT 2 — WHAT YOU'RE ACTUALLY BUYING

Angular gives you per-route decisions.

`RenderMode.Server`

Fresh on every request

Node.js renders HTML on each visit. Data is always current. Needs a running server process.

Home, product listings, category pages

`RenderMode.Prerender`

Built once at deploy time

Static HTML generated at build time via `getPrerenderParams()`. Served from CDN. Data frozen at build.

Blog posts, docs, product detail pages

`RenderMode.Client`

Browser only — no server

Angular renders entirely in the browser. No Node.js involvement. Zero SSR complexity for these routes.

Cart, checkout, dashboard, account

ShopPulse demo: home + listings → Server · product detail → Prerender with getPrerenderParams() · cart → Client. Three answers. One file. Explicit.

SEGMENT 3 — TECHNICAL COMPLEXITY

18 known problems. Four buckets.

BUCKET 1

Two Runtimes, Two Failure Modes

No `window` , `document` , `localStorage` in Node.js

Third-party libraries crash at import time

Fix: `isPlatformBrowser()` · `afterNextRender()` · `inject(DOCUMENT)`

BUCKET 2

The Hydration Contract

Server + client must render identically

`NG0500` mismatch = DOM destroyed + re-rendered = CLS

Fix: `ngSkipHydration` · `@defer` · `withIncrementalHydration()`

BUCKET 3

Double Execution

HTTP calls run on server AND client

Spinner flash on a fully-populated SSR page

Fix: `withHttpTransferCache()` · `TransferState` · `makeStateKey<T>()`

BUCKET 4

Stability

Angular must know when rendering is complete

Open timers/intervals keep the render hanging

Fix: `PendingTasks` · `NgZone.runOutsideAngular()` · `resource()`

SEGMENT 3 — TECHNICAL COMPLEXITY · BUCKET 1

Two runtimes: the browser API problem.

WHAT DOESN'T EXIST IN NODE.JS

- `window`, `document`, `localStorage`, `navigator` — calling any of these in a constructor or `ngOnInit` throws. Server sends nothing. User sees a white page.
- **Third-party libs: the silent risk.** Some access `window` at import time — before you render a single component. Your server crashes before Angular even starts.
- Not a one-time fix. Every `npm install` is a new SSR audit. Chart libs, rich-text editors, analytics SDKs — all suspects.

ANGULAR'S THREE TOOLS

- `isPlatformBrowser()` — conditional guard. Wrap browser-only code. Simple, explicit.
- `inject(DOCUMENT)` — safe DOM reference from `@angular/common`. Works in both environments without crashing.
- `afterNextRender()` — lifecycle hook that never fires on the server. Put browser-only code here; SSR skips it entirely.
- Dynamic `import()` — load browser-only libs lazily, inside a platform check. Prevents crash at module load time.

Case Study #5 in ShopPulse: third-party analytics SDK crashes SSR at import time.

SEGMENT 3 — TECHNICAL COMPLEXITY · BUCKET 2

The hydration contract: what breaks it.

SERVER DOM

```
<nav>
  <div>
    <button>Sign In</button>
  </div>
</nav>
```

no auth state on server

`Date.now() · Math.random()`

3rd-party DOM mutation

MUST MATCH

≠

NG0500

DOM destroyed
re-rendered

↑ CLS spike

CLIENT DOM

```
<nav>
  <div>
    <button>Continue</button>
  </div>
</nav>
```

client reads auth cookie

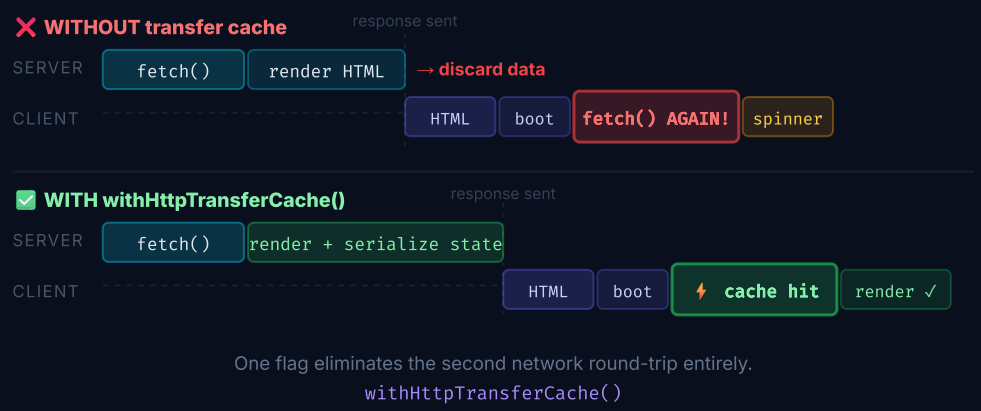
THE FIX SPECTRUM

- `ngSkipHydration` — skip reconciliation for that subtree
- `@defer (on immediate)` — don't SSR this block at all
- `withIncrementalHydration()` — hydrate on viewport / idle / interaction / never

Case Study #1 in ShopPulse: the auth button flicker — the exact production version of this.

SEGMENT 3 — TECHNICAL COMPLEXITY · BUCKET 3

Double execution: one line fix.



One flag eliminates the second network round-trip entirely.
`withHttpTransferCache()`

THE FIX — ONE LINE

```
// app.config.ts
provideClientHydration(
  withEventReplay(),
  withHttpTransferCache()
)
```

SEGMENT 3 — TECHNICAL COMPLEXITY · BUCKET 4

Stability: when does SSR know it's done?

THE PROBLEM

- **Zone.js gave this for free.** It tracked every pending async task. SSR knew exactly when to flush — when the task queue hit zero.
- **Zoneless: no auto-tracking.** Angular can no longer detect when all async work is complete.
- Symptom A: server hangs — an open timer keeps stability pending, request times out. Symptom B: HTML serialised before data resolves — empty template delivered to client.

ANGULAR'S SOLUTION

- `resource()` — auto-registers as a pending task. SSR blocks until the loader resolves. Zero boilerplate.
- `PendingTasks` — explicit registry for custom async work. `run()` wraps async work; SSR waits until it completes.
- `NgZone.runOutsideAngular()` — opt timers and intervals *out* of stability tracking so they don't block the render.

```
// resource() — SSR-safe by default
products = resource({
  loader: () ⇒ fetch('/api/products')
    .then(r ⇒ r.json())
});
```

SEGMENT 4 — THE FULL COST SURFACE

Beyond the obvious. The costs that compound.

Infrastructure

SSR routes need a runtime — static CDN alone is not enough. You need Node.js: long-running process or serverless. `createNodeRequestHandler` for Lambda/Vercel. Cold starts: hundreds of ms to seconds, depending on provider and bundle size. WinterCG handler for edge. Real DevOps overhead.

Team Complexity Tax

Every developer owns the two-runtime mental model. Every library addition is an SSR audit. `withEventReplay()` exists — but only if your team knows why. Onboarding takes longer. This compounds.

The Caching Obligation

`RenderMode.Server` without caching is often slower than optimised CSR. `Cache-Control: stale-while-revalidate` is not optional — it's what makes the LCP benefit real. Most teams skip this step.

Maintenance Surface

A library that passes SSR audit today may access `window` in its next minor version. New class of production incident. `afterNextRender()` + dynamic `import()` are the patterns — but only if someone catches it.

SEGMENT 5 — THE ROI QUESTION

When does the investment actually pay off?

✓ SSR EARNS ITS COST

- ✓ Serves anonymous users at scale
- ✓ Content is meaningful before JS loads
- ✓ TTFB under ~200ms (fast render or cached)
- ✓ LCP is your CrUX bottleneck — not INP
- ✓ Team owns the two-runtime mental model

◆ SSR DOES NOT PAY OFF

- ◆ Auth-gated routes — use `RenderMode.Client`
- ◆ Data changes faster than you can cache
- ◆ INP is the bottleneck — use `@defer` + `withIncrementalHydration()`
- ◆ Stable content on every request — use `RenderMode.Prerender`
- ◆ Team is not ready for two-runtime bugs

Measure first. Open your CrUX data before you open `app.routes.server.ts`

SEGMENT 6 — THE DECISION

Before setting `RenderMode.Server` — ask these.

1. **Does this page have content that matters before JavaScript loads?**
   `If no → RenderMode.Client. Stop here.`

2. **Can you guarantee TTFB under 200ms for this route in production?**
   `If no → RenderMode.Prerender or fix caching first. SSR on a slow server hurts LCP.`

3. **Is your team prepared to own isPlatformBrowser, TransferState, PendingTasks, and the hydration contract?**
   `If no → the complexity tax will compound faster than the performance benefit accrues.`

REFERENCE

The Angular SSR API map.

HYDRATION

- `provideClientHydration()`
- `withEventReplay()`
- `withHttpTransferCache()`
- `withIncrementalHydration()`
- `ngSkipHydration`
- `NG0500`

PLATFORM GUARDS

- `PLATFORM_ID`
- `isPlatformBrowser()`
- `afterNextRender()`
- `inject(DOCUMENT)`
- `dynamic import()`

RENDER & STABILITY

- `RenderMode.Server`
- `RenderMode.Prerender`
- `RenderMode.Client`
- `getPrerenderParams()`
- `PendingTasks`
- `NgZone.runOutsideAngular()`

TRANSFER STATE

`TransferState` `makeStateKey<T>()`

DEFERRED HYDRATION TRIGGERS

`hydrate on viewport` `hydrate on idle` `hydrate on interaction` `hydrate never`

DEMO REFERENCE

ShopPulse — every concept, one app.

LEARN GUIDE

14 chapters · Beginner to Architect Deep Dive · Deployment Topology · SEO Mechanics

ISSUES PAGE

18 common SSR problems · Split problem/fix panels · Category filters · Real Angular API fixes

CASE STUDIES

Auth button flicker · Double-fetch flash · Stale prerendered price · Form input loss · Third-party crash

LIVE DEMOS IN CODE

`@defer` on viewport · idle · interaction · Live stock ticker with Zone.js + signals · Web Vitals panel · TransferState

ROUTE ARCHITECTURE

Server + Prerender + Client modes · `getPrerenderParams()` · Cache-Control per route · `AngularNodeAppEngine`

`RenderMode` **per route.** `withIncrementalHydration()`. `@defer`. `resource()`. `withEventReplay()`.

The tooling is production-grade. The decision still belongs to you — your application, your team, your infrastructure.

And that answer is never the same twice.

NG-INDIA 2026

Thank You

Aprajita Verma · Frontend Architect, Mycom

scan to connect on LinkedIn

⚡ ShopPulse Demo App

📖 Learn SSR Guide

🔬 Case Studies